

Apple platform security: full reading of the accessible corpus

Prepared for David Solis, 3 Oct 2026. Read-only research. No repo changes. Nothing here decides R1 to R5. "Apple states" means a quote or close paraphrase from the cited page. "Hypothesis" means my mapping, not an Apple guarantee.

Reading coverage. I read the whole Apple Platform Security guide sequentially (all 266 pages, the August 2026 PDF, about 9,870 lines of extracted text, in chunks with running notes) and every article listed below, except the access gaps in the last table. The gaps are real: some Private Cloud Compute (PCC) guide pages did not load, one article was cut off near its end, and older guide editions were not read. I do not claim to have read anything marked as a gap. The guide names the PKA hardware proof; Apple's corecrypto post names a second formal-verification result, so "formal verification" below is per component, never a claim about Apple platforms as a whole.

1. Corpus and coverage

Item	Edition	Read	URL
Apple Platform Security guide (Apple calls it a "guide")	PDF Aug 2026	All 266 pages, sequential	https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf
Memory Integrity Enforcement	Sep 2025	Full	https://security.apple.com/blog/memory-integrity-enforcement/
kalloc_type (XNU memory safety)	2022	Full except the last portion (gap G2)	https://security.apple.com/blog/towards-the-next-generation-of-xnu-memory-safety/
What if we had SockPuppet in iOS 16?	2022	Full	https://security.apple.com/blog/what-if-we-had-sockpuppet-in-ios16/
Private Cloud Compute blog; PCC security research; Expanding PCC	2024 to 2026	Full	https://security.apple.com/blog/private-cloud-compute/ , /pcc-security-research/ , /expanding-pcc/
PCC guide: Anticipating Attacks; Verifiable Transparency; Inspecting Releases; Hardware Integrity	Current	Full (four pages)	https://security.apple.com/documentation/private-cloud-compute/attacks (and /verifiabletransparency , /inspectingreleases , /hardwareintegrity)
A blueprint for formal verification of Apple corecrypto	Current	Full	https://security.apple.com/blog/formal-verification-corecrypto/
iMessage with PQ3	Feb 2024	Full	https://security.apple.com/blog/imessage-pq3/
iMessage Contact Key Verification	2023	Full	https://security.apple.com/blog/imessage-contact-key-verification/
Apple Security Bounty: "Upgraded", "Evolved"; Research Device Program 2024	2022 to 2025	Full (program posts, little engineering content)	https://security.apple.com/blog/apple-security-bounty-evolved/ and sibling posts
Outside source: Project Zero on iMessage/BlastDoor	Jan 2021	Full (not Apple)	https://projectzero.google/2021/01/a-look-at-imessage-in-ios-14.html

Apple's blog index lists 12 posts; I read all 12 (kalloc_type except its tail, gap G2). I found no other Apple security "white papers". That is a statement about what I could find, not proof that none exist.

2. Principles Apple documents (with source)

Principle	What Apple states, and where
Fail closed	Guide: the system volume hash is checked on read; PAC faults abort; IOMMUs default to deny; if OCSP is unavailable, trust evaluation fails; App Transport Security hard-fails invalid certificates; no boot without a valid LocalPolicy; iCloud Keychain escrow destroys the record after 10 failed attempts; Notes rejects unsupported attachments. PCC attestation includes only verified properties; the guide names the freshness code as the exception.
Anti-replay, epochs, rollback	Guide: a new Authorization Random value ends old cards; payment cryptograms carry counters and server anti-replay values; LocalPolicy uses nonces and a security epoch; ADP sets minimum OS versions so old clients cannot repair state; the Digital Credentials API uses an anti-replay value. PQ3: each message key is deleted on decrypt, which "provides protection against replayed messages".
Least privilege and separation	Guide: extensions run in separate address spaces; keyboards in a no-network sandbox; HomeKit camera frames render in a separate process; ReplayKit capture is out of process; the network processor has its own PCIe bus with an IOMMU; DriverKit moves drivers to user space; SiriKit passes only the relevant fragment. PCC: process isolation, one egress process, per-request workers. Bounty post: GPU-process architecture and IPC hardening removed "100 percent of the IOUserClient attack surface from the WebContent sandbox".
Isolate data from control	kalloc_type: pure-data allocations are kept apart from pointer-bearing ones, to mitigate use-after-free early in an exploit chain. Compiler diagnostics (-Werror) drove adoption to about 95% of the kernel in iOS 16. Apple's own caveat: partial adoption leaves the unenrolled targets, so coverage matters.
Probabilistic, layered mitigation	SockPuppet post: Apple estimates a single-strategy exploit succeeds about 8% of the time under kalloc_type, that the best possible exploit is capped near 92%, and near 59% from an unprivileged position. These are Apple's estimates, not measurements. Apple also lists what stays attackable: signature collisions, the data submap, within-type use-after-free.

Remove attack surface	Guide: Intel ME stripped; unnecessary tools not included; Lockdown Mode; "the best way to manage passwords is not to have to use one". PCC: no shell, no general-purpose logging.
Structured inputs, memory-safe parsing	PCC uses Protobuf-style structured formats for external input, memory-safe Swift for first processing of untrusted input, and a BlastDoor-derived sandbox for media decode. Apple admits PCC sandboxes are coarser than WebKit's or BlastDoor's. MIE post: Swift is used "for new code and targeted component rewrites".
Bound lifetime of state and keys	Guide and PCC: per-request keys, ephemeral data mode, deleteAfterRead, recycling of long-lived processes (bounded, not immediate wipe). PQ3: keys have timestamps that limit validity; post-quantum rekey about every 50 messages and at least every 7 days.
Safe write ordering	Guide: APFS atomic safe-save; Notes creates the encrypted record before deleting the original; Shared iPad evicts a user only after cloud sync completes, otherwise sign-in fails.
Selective disclosure	Guide (Wallet ID): per-element antiforgery lets a person share only the requested fields; consent sheets show who, what, retention; the web path uses a signed request and encrypted response.
Transparency and verifiability	PCC: append-only log with Merkle inclusion proofs, devices send only to attested and logged software, releases published within 90 days, hardware ceremony with three Apple organizations plus an independent SOC 3 auditor. Contact Key Verification: devices check inclusion proofs and consistency of an append-only log on the device, and gossip log hashes to detect split views. PQ3 binds each signature to message context fields, including position in the ratchet.
Test the design by attacking it	MIE post: Apple's offensive team attacked it against real exploit chains from 2020 to 2025. PCC: a research environment and bounty. Bounty posts: Target Flags let researchers prove capability programmatically.

3. What is formally verified (per component, with stated limits)

Component	Apple states
Public key accelerator (PKA) hardware	Guide: "Starting with A13 SoCs, the PKA's encryption implementations have been proved to be mathematically correct using formal verification techniques." The corecrypto post says Apple has used formal verification in silicon for 15+ years and on the PKA since 2019.
corecrypto ML-KEM and ML-DSA	Over 50,000 Isabelle proof steps using Cryptol, SAW and Isabelle, covering C and ARM64. It found a real missing-step bug and an error in a third-party proof. Stated limits: assumes the compiler is correct; SAW did not cover all ML-DSA message sizes (conventional testing covers them); scope is functional correctness, with leakage covered by testing and simulation. Apple: the strongest assurance "comes from combining formal verification with conventional methods and critically evaluating the end-to-end results."
iMessage PQ3 protocol	Machine-checked Tamarin proof (ETH Zurich) and game-based proofs (Stebila), conditional on the underlying algorithms staying secure. This covers the protocol model. A separate third-party code review "found no security issues". The earlier 2019 protocol changes were verified by symbolic evaluation.

Reading: Apple applies proof to specific, small, well-specified components and pairs it with testing and review. I found no Apple statement that the whole platform is formally verified, and I would not write one.

4. Limits Apple states about itself

- "App Store security measures alone can never be perfect." Protections "aren't engineered to protect against choices a user might be tricked into making". macOS users "are free to operate within the security model ... including running completely unsigned and untrusted code".
- MIE: "there's no such thing as perfect security"; some issues survive it.
- UEFI is described as a "hardware architectural limitation". Weak passwords weaken backup keybags. Convergent-encryption metadata is not end-to-end under ADP. Express Mode/Transit lets nearby readers access some history. Universal Clipboard is shared by default. Deprecated Wi-Fi, TLS and profile crypto is kept for compatibility.
- PCC: observability filtering "can't completely eliminate the risk"; homogeneity means one software attack could scale; published source lacks reproducible builds, so it "cannot be proven that published PCC binaries are compiled from published source"; the research VM uses a paravirtualized GPU and virtual SEP; the Google Cloud deployment is "gradually ramping towards the complete set of protections".
- PQ3: authentication keys stay classical; the "Level 0 to 3" scale and "state of the art" claims are Apple's own comparison. Contact Key Verification: some inconsistencies "can be detected only by third-party auditors".
- BlastDoor (Project Zero, outside view): a real obstacle, not a proof. Lockdown Mode "makes tradeoffs".

5. Hypotheses: mapping to Tetris.js (not Apple guarantees)

These are my suggestions for discussion. None is approved, none is a ruling, and none implies a rewrite or a tool choice.

Apple habit	Possible JavaScript counterpart
Small parser, validated output, structured inputs	One narrow place where saved data enters; it returns a valid state or rejects. JS is memory-safe by language, so the risk there is invalid state, not memory corruption.
Fail closed; validity before use	Reject unknown or out-of-range state rather than repair it silently.
Least privilege, narrow interfaces	Engine gets no browser access; renderer, storage and input get read access or submit actions only.
Anti-replay, monotonic counters	Version or sequence fields on saved state and replays, so stale data cannot reapply.
Safe write ordering	Write the new save fully before replacing the old one.
Control vs data; exhaustive adoption	Keep inert data apart from executable paths. Apple's point that partial adoption leaves targets argues for checking a rule everywhere or not claiming it (for example a lint rule that covers all files).

Remove surface	Fewer features, fewer dependencies; this fits your minimal-complexity goal.
Proof where tractable, plus testing	Prove the small core model under stated assumptions; keep adversarial tests and mutation results for what the model leaves out. Say "proved for model M under assumptions A", never "unhackable".
Published limits	Write the threat model with its boundaries: a player controls their own browser, so the claim is correctness and containment, not that it cannot be tampered with.

Not carried over: hardware features (PAC, memory tagging, Secure Enclave), kernel allocators, code signing, attestation hardware. They depend on platforms you do not control.

6. Access gaps (what I did not read)

ID	Gap	Recovery tried
G1	PCC guide pages Request Flow, Request Handling, Software Layering, Stateless and Enforceable, and the guide index returned an error or empty page (script-rendered). Other PCC pages cited inside "Anticipating Attacks" (attested request handling, transitive trust, node selection, management) were not reachable.	Refetch with and without a trailing slash; search for alternative URLs. Not recovered.
G2	kalloc_type post: the page text is capped at about 50,000 characters and ends mid-sentence in the randomized-bucketing section. The limitations and conclusion at the end were not read.	Refetch; search excerpts only. Not recovered.
G3	Older iOS Security Guide editions (for example 2014 and 2019 archives) were not read; I know only that they exist.	Not attempted beyond locating them.
G4	Edition mismatch: the PDF carries August 2026 (history lists "Terminal and script protections", "Search on iCloud.com security", "SharePlay security", "Nearby sharing security"), but the web version I opened earlier showed March 2026. I read the PDF.	Not resolved. The PDF is treated as the latest.
G5	Bounty categories and guidelines pages were found but not read; they are program rules, not engineering.	Listed only.